

Documento de Análisis y Diseño de Recetario inteligente con adaptabilidad por cantidad de materiales y personas

Fecha [01-07-26]

Versión [1.2]

Integrantes Catalina Florencia Sáez González
del grupo: Rodrigo Andrés Rosales Arias
Aníbal Ignacio González Chiguay

1 Introducción

1.1 Diagnóstico e identificación del problema, desafío u oportunidad

Dentro de la vida cotidiana hay una extensa cantidad de tareas a realizar, y una de las más fundamentales es la alimentación. Esta acción no solo se refiere a la propia de cada uno de nosotros, sino que es una situación que se extiende a la vida social, por ejemplo, al momento de organizar algún tipo de reunión, almuerzo, cena, etc. Con relación a lo anterior, nace un problema que es más común de lo que se reconoce, las listas y recetas de alimentos, o sea lo relacionado a la logística de un evento. Este proyecto está enfocado en usuarios específicos: por un lado, anfitriones de casa que organizan reuniones y, por otro lado, estudiantes universitarios que comparten arriendo o viven en residencias y necesitan optimizar sus compras semanales. Dado que este tipo de usuarios debe considerar un presupuesto, la cantidad de personas y materiales que se deben utilizar, la gestión de todo lo ya mencionado puede volverse tediosa, lo que puede provocar errores o malos cálculos que podrían tener un impacto negativo tanto en la persona como en la receta. Pero ¿Y si un programa hiciera los cálculos de cantidad y presupuestos necesarios?

Evolucionando de mejor manera nuestro proyecto, desde un simple escalador de recetas hemos pasado a un Planificador Gastronómico Mensual, aunque sigue teniendo su función principal de escalar las recetas que desee el usuario, esta evolución permite no solo escalar porciones, sino organizar la logística semanal/mensual mediante un calendario interactivo y reportes de gestión, optimizando tanto el tiempo del usuario como su presupuesto.

1.2 Propuesta de solución

Al ser un problema común se necesita tener una solución flexible que permita solventar el problema en todas sus variaciones, como debe ser flexible debemos ocupar un elemento que también lo sea, como un software y un lenguaje de programación de la misma naturaleza. C++ es uno de los lenguajes más flexibles para trabajar software dinámico, ya que nos permite trabajar de mejor manera los recursos como la memoria, bibliotecas de funciones, estructuras de variables y módulos. El software contaría con una arquitectura de programación orientada a objetos (POO). La POO es clave para desarrollar la solución que consiste en un recetario inteligente con adaptabilidad por cantidad de materiales y personas. El proyecto no es un repositorio de texto personal, es un motor de cálculo dinámico que permite que las listas de materiales necesarios y sus precios sean escalados para una cantidad N de personas. El hecho de agregar una receta es solo un extra, por esto, resolvería el problema de error humano al momento de realizar estimaciones y cálculos los cuales son fundamentales para cada anfitrión o estudiante y su bolsillo.

2 Requisitos del Sistema

2.1 Requisitos Funcionales

RF1 [Registro de recetas. El sistema debe permitir al usuario ingresar nuevas recetas, especificando el nombre de la nueva receta, la cantidad de porciones base, y el listado de ingredientes con sus respectivas cantidades.]

- RF2 [Cálculo de ingredientes. El sistema debe recalcular automáticamente las cantidades de cada ingrediente cuando el usuario modifique el número de porciones finales (N personas).]
- RF3 [Visualización de listado. El sistema debe mostrar de forma clara y ordenada la lista de recetas almacenadas y, al seleccionar una, desplegar los detalles técnicos e ingredientes.]
- RF4 [Estimación de presupuesto. El sistema debe permitir asociar costos a los materiales mediante el ingreso manual del valor base por ingrediente al momento de crear la receta, para luego generar un presupuesto total estimado basado en el escalonado de la receta (es decir, mostrar el precio total de la receta, escalonando proporcionalmente de acuerdo con las porciones).]
- RF5 [Validación de entradas. El sistema debe validar los datos ingresados por el usuario, asegurando que las cantidades, precios y número de porciones sean valores numéricos válidos mayores a cero, evitando que el programa falle por errores de tipeo.]
- RF6 [Autenticación y gestión de usuarios. El sistema debe contar con una pantalla de inicio de sesión (Log in) que permita crear y autenticar distintos perfiles de usuario, para que cada persona mantenga su propio repositorio privado de recetas.]
- RF7 [Planificación mensual. El sistema debe contar con un calendario interactivo que permita al usuario asignar recetas específicas a fechas del mes.]
- RF8 [Análisis de perfil nutricional. El sistema debe clasificar los ingredientes adquiridos según su categoría en una pirámide alimenticia (Base, Proteínas, Granos, entre otros), calculando y visualizando mediante gráficos la distribución mensual del consumo, facilitando el seguimiento de los hábitos alimenticios.]

2.2 Requisitos no funcionales

- RNF1 [Usabilidad. La interfaz desarrollada en Qt debe ser amigable y fácil de usar, permitiendo que un usuario sin conocimientos técnicos pueda realizar cálculos de logística rápidamente.]
- RNF2 [Portabilidad y compatibilidad. Gracias al uso de C++ y Qt, el sistema debe ser capaz de ejecutarse en entornos como GNU/Linux (vía WSL o nativo) asegurando el mismo comportamiento.]
- RNF3 [Eficiencia en el manejo de recursos. El software debe hacer un uso eficiente de la memoria y el procesamiento, garantizando que el recálculo de cantidades sea instantáneo para el usuario.]
- RNF4 [Mantenibilidad del código. El proyecto debe seguir una arquitectura de programación orientada a objetos, facilitando la corrección de errores o la adición de futuras funcionalidades como módulos independientes.]
- RNF5 [Interfaz de usuario interactiva. El sistema debe proporcionar una interfaz gráfica fluida que conecte las funciones de edición, visualización y cálculo, manteniendo coherencia visual en todas sus ventanas.]

2.3 Requisitos de Ambiente:

2.3.1 Hardware de Desarrollo:

- Computadores personales: Equipos con arquitectura de 64 bits que soporten el IDE y el compilador sin sobrecarga térmica o de rendimiento.
- Memoria RAM: Mínimo 4 GB, aunque se recomiendan 8 GB para mantener fluido el entorno de desarrollo y gestionar eficientemente la memoria dinámica y la estructura de vectores (`std::vector`) al compilar y hacer pruebas de estrés en C++.
- Unidad de procesamiento (CPU): Procesador multinúcleo (mínimo 2 núcleos, ej. Intel Core i3, AMD Ryzen 3 o equivalentes) para compilar la interfaz visual y procesar el algoritmo de cálculo instantáneo.

2.3.2 Software de Desarrollo:

- C++ estándar: Aprovecharemos la capacidad de ser un lenguaje flexible e ideal para POO, permitiéndonos que los datos se manejen de manera eficiente.
- Qt: Framework que nos permite tener una interfaz interactiva para así tener una experiencia intuitiva y amigable para los usuarios sin necesidad de tener conocimientos técnicos.
- Entorno GNU/Linux: Entorno de desarrollo para utilizar herramientas nativas de debugging y compilación (como g++). Esto se trabajará de forma nativa en distribuciones como Linux Mint administrando configuraciones desde la terminal, o bien a través de virtualización mediante WSL.

2.4 Funcionalidades fuera de alcance:

Esta sección tiene como objetivo delimitar el alcance real de la solución actual, donde se especificarán algunos elementos y funcionalidades que no forman parte de la entrega. La (s) funcionalidad (es) a continuación fueron postergadas para priorizar la estabilidad del motor de cálculo principal o por restricciones de tiempo, pueden ser implementadas o no en un futuro desarrollo.

2.4.1 Implementación con API de supermercado para cotización y presupuestos en tiempo real

La idea surgió en la mitad del desarrollo, buscando mayor complejidad y añadir un valor agregado distintivo. Las APIs de supermercados habría escalado a un nivel mayor del motor de cálculo del proyecto e incorporando mayores y gráficos estadísticos más avanzados e incorporando la estimación de presupuesto.

La decisión de que quedara fuera del alcance del proyecto son las siguientes razones:

- Restricción de accesibilidad: Las interfaces de programación (APIs) de las grandes empresas de retail no son accesibles gratuitamente ni son prestadas para proyectos académicos de índole similar, requieren convenios privados, contratos o credenciales de acceso.
- Visión a futuro: Aunque en el estado actual del proyecto no se integrara es completamente viable en el caso de que una corporación o empresa que posea el acceso legítimo a dichas APIs se interesara en el proyecto, otorgándole un valor agregado y competitivo en mercado.

3 Planificación del Proyecto

3.1 Objetivo General (Goal)

- Desarrollar una interfaz interactiva en C++ utilizando el framework Qt, que permita la gestión, almacenamiento y escalado dinámico de recetas de cocina para optimizar la planificación logística y presupuestaria en situaciones comunes como lo son el cocinar día a día o en eventos sociales.

3.2 Objetivos Específicos

OE1. [Diseñar un programa en C++ que permita al usuario ingresar datos para ser almacenados]

OE2. [Determinar una manera sencilla de guardar los datos de cada usuario]

OE3. [Implementar una interfaz interactiva al código en C++ con Qt]

OE4. [Implementar un módulo de gestión temporal y análisis de datos mediante gráficos estadísticos para el control de gastos y nutrición.]

3.3 Actividades y Milestones

[Enumerar qué actividades debe realizar para dar cumplimiento a cada objetivo, y cada tarea que da cumplimiento a cada actividad. También debe agregar el desarrollo de actividades y tareas de gestión del proyecto.]

	Responsable	Participación Equipo	Fecha Comienzo	Fecha término
[Nombre de elemento (objetivo, actividad de objetivo, actividad de gestión (no resuelve un objetivo en específico), tarea, milestone)]	[Responsable de avance del elemento.]	[% de participación de los miembros del equipo en la actividad]	[Fecha de comienzo del elemento. Si aún no comenzado, poner fecha tentativa de comienzo].	[Fecha de término del elemento. Si no terminado, poner fecha tentativa de término]
Milestone 1: Finalización de documentos (Primera versión) (Hito 0)	Catalina Sáez Rodrigo Rosales Aníbal González	100%	20-04-2026	30-04-2026
A1.1 Diseño e implementación de las clases Ingrediente y Receta en C++ para gestionar nombres, cantidades y unidades.	Aníbal González	100%	30-04-2026	07-05-2026

A1.2 Implementación de la lógica matemática y algoritmo de estimación de presupuesto que multiplica cantidades base por el factor de porciones seleccionado	Rodrigo Rosales	100%	07-05-2026	15-05-2026
A2.1 Desarrollo de funciones de persistencia que permitan abrir, leer y guardar recetas en formato txt.	Catalina Sáez	100%	15-05-2026	25-05-2026
A2.2 Implementación de validación de entradas numéricas para evitar errores de tipeo del usuario al ingresar datos.	Catalina Sáez	100%	25-05-2026	01-06-2026
Milestone 2 (Hito 1): Entrega del código fuente en C++ que procesa recetas y guarda datos, verificado por consola.	Catalina Sáez Rodrigo Rosales Aníbal González	100%	01-06-2026	12-06-2026
A3.1 Creación visual de las pantallas mínimas en Qt: menú principal, agregar receta y ver receta.	Anibal González	100%	01-06-2026	07-06-2026
A3.2 Creación visual de las interfaces secundarias: calcular porciones.	Catalina Sáez	100%	07-06-2026	13-06-2026
A3.3 Desarrollo de módulo de reportes gráficos con Qt Charts (Gráficas de porcentaje de vitaminas por receta).	Aníbal González	100%	20-06-2026	26-06-2026
A3.4 Conexión de los botones de la interfaz con las funciones de cálculo y almacenamiento. desarrolladas previamente.	Aníbal González	100%	26-06-2026	28-06-2026
A4.1 Ejecución de pruebas unitarias y de integración para detectar fallas en el ingreso de	Rodrigo Rosales	100%	28-06-2026	30-06-2026

datos o en el guardado de archivos.				
A4.2 Ajustes finales en el diseño Qt para mejorar la navegación del usuario y corregir bugs visuales.	Catalina Sáez Rodrigo Rosales Aníbal González	100%	30-06-2026	02-07-2026
Milestone 3 (Hito 2): Entrega de la aplicación con manual de usuario y video demostrativo de funcionamiento.	Catalina Sáez Rodrigo Rosales Aníbal González	100%	29-06-2026	03-07-2026

3.4 Carta Gantt

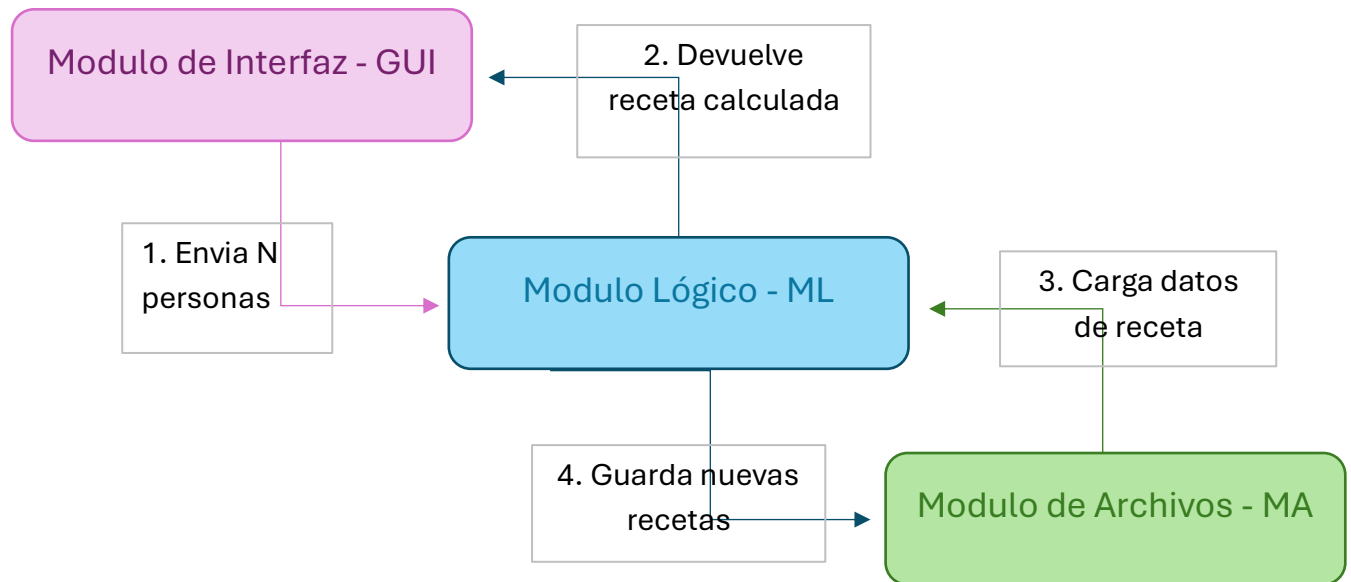
[Descripción gráfica de los tiempos planificados (por semana) por actividad, tarea y logro de milestone. Para aquellas cumplidas, se pone el tiempo real empleado.]

2026												
	Ene	Feb	Mar	Abr	May	Jun	Jul	Ago	Sep	Oct	Nov	Dic
Hito 0												
A1.1												
A1.2												
A2.1												
Hito 1												
A3.1												
A3.2												
A3.3												
A3.4												
A4.1												
A4.2												
Hito 2												

4 Diseño de Arquitectura del Sistema

4.1 Diagrama de Arquitectura

[La siguiente figura muestra los módulos internos del sistema y sus principales interacciones. Los módulos aparecen indicados con su nombre corto o abreviatura y se deben indicar los flujos principales de información entre los módulos.]



4.2 Enumeración de Módulos

La tabla muestra los módulos de la Figura 2. Por cada módulo se entrega un breve párrafo descriptivo de su propósito, además de la sección en donde se especifica el módulo en detalle.

Tabla: Módulos de la arquitectura del sistema

Módulo	Propósito
Módulo de Interfaz (GUI)	Gestiona la capa visual desarrollada en Qt. Su función principal es capturar las interacciones del usuario (clics en botones, ingreso de texto) y desplegar la información de las recetas de forma gráfica e intuitiva.
Módulo Lógico (ML)	Representa el “cerebro” del sistema, implementado mediante Programación Orientada a Objetos en C++. Contiene las clases Receta e Ingrediente y se encarga de ejecutar el algoritmo de escalado proporcional de las cantidades según el número de personas solicitado.
Módulo de Archivos (MA)	Es el componente responsable de la persistencia de datos. Se comunica con el sistema operativo para leer archivos de texto (.txt) que contienen las recetas guardadas y las credenciales de inicio de

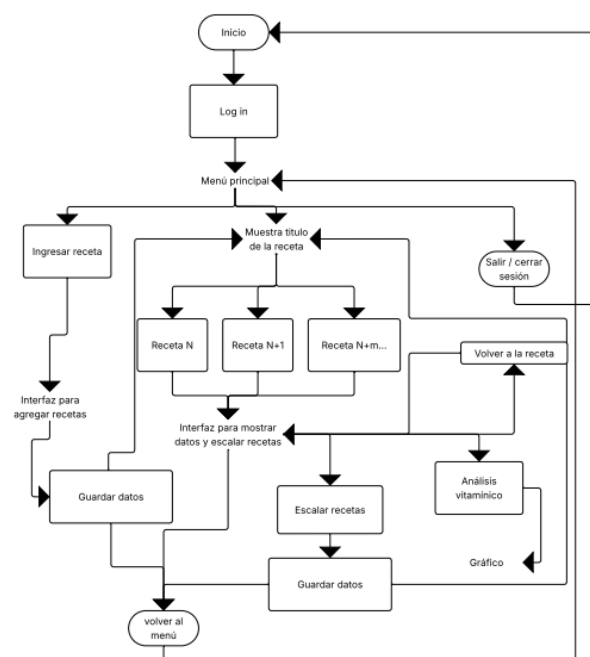
	sesión, permitiendo cargar el perfil correcto al momento de hacer “Log in” y asegurando que la información de cada usuario se mantenga privada y no se pierda al cerrar el programa.
--	--

4.3 Matriz de Requisitos Funcionales y Módulos

Requisito Funcional	GUI	ML	MA
RF1: Registro de Recetas	X	X	X
RF2: Cálculo de Ingredientes		X	
RF3: Visualización de Listado	X		X
RF4: Estimación de Presupuesto		X	
RF5: Validación de entradas	X		
RF6: Autentificación y gestión de usuarios	X		X
RF7: Análisis de Perfil Nutricional	X	X	

5 Diseño de Interfaces

5.1 Modelo de Navegación



5.2 Prototipos de Interfaces Usuarias

[Repetir para las GUI principales del sistema]

Prototipo [XX]

[Mostrar gráficamente el diseño de la interfaz (no funcional). El nombre referenciado debe ser coherente con el nombre correspondiente especificado en 3.1. Detallar los eventos que se pueden gatillar desde la interfaz, completando la siguiente tabla:]

Evento	Interacción	Acción	Objeto afectado
[Log in]	[Hacer clic en el botón de inicio].	[Se muestra la interfaz principal, la cual contendrá los eventos principales].	[Interfaz de inicio log in].
[Ingresar receta]	[Presionar el botón "+".].	[Al ser seleccionada realizará el proceso abrir para editar y agregar el nuevo contenido].	[Interfaz de edición].
[Guardar datos]	[Hacer clic en el icono o botón "Guardar"].	[Al momento de ser presionado guardará los datos que el usuario haya modificado y mostrará el título que haya escogido el usuario en el menú de inicio].	[Crea un archivo “.txt” que contendrá la información actualizada y crea un objeto en el menú principal con el título del archivo].
[Receta N]	[Seleccionar una receta de la lista].	[Al ser presionada mostrará los datos actualizados por última vez a través de una interfaz que contiene el próximo evento].	[Objeto creado que permite visualizar la lista completa].
[Escalar recetas]	[Ingresar número de personas y presionar "Calcular"].	[Multiplica la cantidad de material para la cantidad de personas que el usuario desee].	[La información de la receta correspondiente a la cantidad es cambiada].
[Volver al menú]	[Presionar el botón "Atrás" o "Volver"].	[Lleva al usuario directo al menú principal].	[Cierra la interfaz en la que se encuentra el usuario y abre el menú principal].
[Salir / Cerrar sesión]	[Presionar el botón "Salir"].	[Cierra sesión transportando al usuario nuevamente al proceso]	[Reinicia el inicio de sesión para agregar un nuevo usuario sin borrar nada del usuario anterior].

de “Log in” para iniciar una nueva sesión].			
[Mostrar gráfico de vitaminas por receta]	[Presionar el botón “Análisis calórico”]	[Genera un gráfico de “torta” con las vitaminas que más contiene la receta]	[El gráfico]

5.3 Prototipos visuales.

Login

A screenshot of a 'Dialog' window titled 'Dialog'. It contains two text input fields labeled 'Usuario' and 'Contraseña', and a blue button labeled 'Ingresar' positioned below the password field.

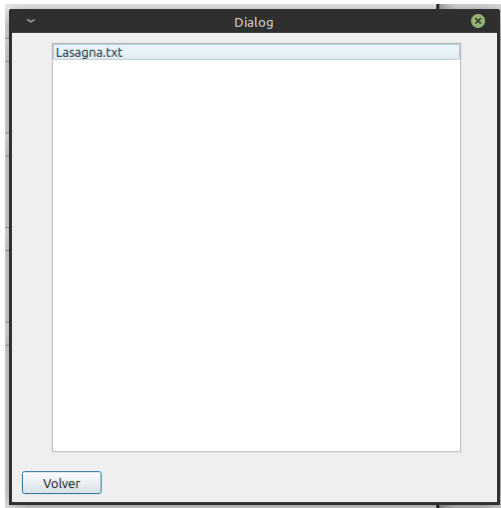
Ventana Principal

A screenshot of a 'MainWindow' window titled 'MainWindow'. It features four buttons arranged vertically: 'Agregar Receta', 'Ver Recetas', 'Presupuesto y Precios', and 'Cerrar Sesión'.

Agregar receta (3 clases)

Three sequential 'Dialog' windows showing the steps to add a recipe.
 1. The first dialog asks for 'Nombre de la receta' (text input) and 'Número de porciones' (spin box set to 0), with a 'Siguiente' button.
 2. The second dialog has a table with columns 'Ingrediente', 'Cantidad' (spin box at 0.00), and 'Unidad' (dropdown set to 'gr'). It includes an 'Agregar' button and a 'Siguiente' button at the bottom.
 3. The third dialog asks '¿Cómo se prepara?' with a text area for 'Escribe acá:', a '+ Agregar paso' button, and a '¡Guardar Receta!' button at the bottom.

Ver recetas



Enlace al repositorio GitHub con evidencia:

- <https://github.com/crxtalyy/Recetario-Inteligente-TEL20125>